

Gas Injury Detection via Multi-Channel Gas Sensor Array and Machine Learning

Model Study Report

Brent Hoang

May 7, 2026

Dataset

Data was collected across 18 sessions using an ESP32-based sensor array with 6 electrochemical channels: VOC, NH₃, HCHO, H₂S, EtOH, and RH. 16 sessions were used for training and 2 were held out as unseen test data.

Each session was manually labelled into three classes: **Baseline** (no sample), **Sweat**, and **Blood**. Blood sessions contain only baseline and blood regions; sweat sessions contain only baseline and sweat regions.

Table 1: Dataset summary.

Property	Count
Total sessions	18 (10 blood, 8 sweat)
Training sessions	16
Held-out test sessions	2 (1 blood, 1 sweat)
Training windows (balanced)	572
Test windows	125

Methodology

Feature Engineering

Causal helper columns were computed from the 6 raw channels to give models access to temporal context without look-ahead bias:

Table 2: Engineered feature groups (per channel, 6 channels total).

Group	Description	Count
Raw signal	Direct sensor reading	6
Rate of change (<code>_roc</code>)	<code>diff()</code> — instantaneous trend	6
Acceleration (<code>_acc</code>)	<code>diff()</code> of roc	6
Rolling mean (15 / 30 / 60 s)	Smoothed level at 3 timescales	18
Rolling std (15 / 30 / 60 s)	Local volatility	18
Rolling roc (15 / 30 / 60 s)	Smoothed absolute rate of change	18
Total		72

Random Forest / XGBoost features: each 60s non-overlapping window is summarised into 4 aggregates (mean, std, max, slope) $\times$ 72 columns = **288 features** per window.

CNN / TCN features: a 30s sliding window (stride 5s) retains the raw time-series with **24 channels** (raw + `_roc` + `_roll_mean_30` + `_roll_std_30`) giving each model explicit trend, level, and volatility context.

Train / Validation Split

Splits are performed at the *session level* so all windows from a given session stay together, preventing temporally correlated windows from leaking across the boundary. An 80/20 stratified split was used across the 16 training sessions; 2 sessions were held out as unseen test data throughout the entire study.

Model Selection

Four model families were evaluated. Each was chosen to test a different trade-off between interpretability, temporal modelling capacity, and suitability for embedded deployment.

Table 3: Model characteristics.

Model	Strengths	Limitations
Random Forest	Interpretable feature importances; robust on small tabular datasets; no scaling required.	Requires hand-crafted window aggregates; does not learn from raw time-series directly.
XGBoost	Gradient boosting corrects residual errors sequentially, typically outperforming RF on the same features; built-in regularisation.	Same tabular feature dependency as RF; less interpretable than RF.
1D CNN	Learns temporal patterns directly from raw signal; no manual feature engineering needed; parallelisable and fast to train.	Requires more data to generalise well; local receptive field may miss longer-range dependencies.
TCN	Dilated causal convolutions give an exponentially wider receptive field with fewer parameters than stacked Conv1D; residual connections stabilise training.	Slightly more complex to tune than CNN; still data-hungry compared to tree-based methods.

Feature importances from the Random Forest identified HCHO and RH as the dominant discriminators between classes, which informed the channel selection for the CNN and TCN input.

Results

Validation Accuracy

Table 4: Validation accuracy on held-in sessions (115 windows).

Run	Model	Val acc
Run 1	RF (50 features)	99.1%
Run 2	RF (top-20)	98.3%
Run 3	RF (all 288)	99.1%
Run 7	XGBoost	99.1%
Run 4	CNN R1	99.4%
Run 5	CNN R2	99.6%
Run 6	TCN R1	99.5%
Run 6	TCN R2	99.6%

Test Accuracy on Held-Out Sessions

Table 5: Test accuracy on `blood_9` and `sweat_7` (never seen during training).

Run	Model	<code>blood_9</code>	<code>sweat_7</code>
Run 1	RF — 50 features	100.0%	87.9%
Run 2	RF — top-20	100.0%	89.7%
Run 3	RF — all 288	100.0%	100.0%
Run 7	XGBoost	100.0%	100.0%
Run 4	CNN R1 (3-layer)	99.3%	100.0%
Run 5	CNN R2 (2-layer)	100.0%	100.0%
Run 6	TCN R1	98.5%	100.0%
Run 6	TCN R2 (reduced)	99.3%	96.6%

CNN R2 and XGBoost achieve the best overall result: 100% accuracy on both held-out test sessions. RF Run 1 underperforms on `sweat_7` (87.9%), likely because the feature selection threshold discards several sweat-relevant features; adding all features (Run 3) recovers the gap.

Sanity Check

Two tests were conducted to verify that strong test results reflect genuine learned patterns rather than a lucky data split or structural data artefacts.

Test 1 — Seed Sweep

The session split was repeated with 5 random seeds {42, 0, 7, 123, 99}. CNN R2 and TCN R2 were retrained from scratch for each seed and evaluated on the fixed test sessions.

Table 6: Seed sweep results — mean $\pm$ std across 5 seeds.

Model	Session	Mean	Std	Min	Max
CNN R2	<code>blood_9</code>	99.4%	1.2%	97.0%	100.0%
CNN R2	<code>sweat_7</code>	99.7%	0.7%	98.3%	100.0%
TCN R2	<code>blood_9</code>	99.4%	0.6%	98.5%	100.0%
TCN R2	<code>sweat_7</code>	99.5%	0.7%	98.3%	100.0%

Verdict: PASS. Standard deviation $\leq 1.2\%$ across all seeds. Results are not dependent on a particular train/val split.

Test 2 — Permutation Test

Training labels were randomly shuffled 5 times. A new model was fitted on scrambled labels each time and evaluated on the real test labels.

Table 7: Permutation test results — permuted mean vs real accuracy.

Model	Session	Perm mean	Real acc	Gap	Verdict
CNN R2	blood_9	34.1%	100.0%	+65.9%	PASS
CNN R2	sweat_7	49.7%	100.0%	+50.3%	PASS
TCN R2	blood_9	22.5%	99.3%	+76.7%	PASS
TCN R2	sweat_7	36.2%	96.6%	+60.3%	PASS

Verdict: **PASS.** Permuted accuracy collapses to near-random ($\sim 33\%$ expected for 3 classes). The real models outperform by 50–77 percentage points, confirming they learn genuine gas signal patterns rather than exploiting session structure or data artefacts.